JMeter Performance Analysis – ParaBank

Group 35 · Assignment 3, Activity 1 (Task 2: execution and analysis)
Prepared by Adithya Kaparthi (s4160686) · Test plan (Task 1) by Praneeth (s4139595)
Test plan: `JMeter/jmeterplan-35.jmx` · Runs executed 15 October 2026, 15:40–16:20 AEDT

1. Introduction

This report presents the results of performance testing the ParaBank demo banking site (`https://parabank.parasoft.com`) with Apache JMeter. One user journey was tested under four loads (1, 10, 20 and 50 concurrent virtual users), and for each load we recorded Throughput, Average, Min, Max, 95th percentile and Error %. The report compares the four loads and explains why performance changes as load increases.

The tested scenario is: **Login → Accounts Overview → View Account Details → Transfer Funds → View Transactions → Logout**, with a 300 ms delay before the home page is loaded.

2. Test setup

2.1 Environment

Item	Value
Load tool	Apache JMeter 5.6.3 (GUI mode), Java 25, Windows 11
Target	parabank.parasoft.com over HTTPS, served through Cloudflare (Melbourne edge, `CF-RAY …-MEL`)
Client location	Single laptop, home internet connection, Australia (all virtual users share one IP address)
Date / time	15 Oct 2026: TG1 ≈15:40, TG2 ≈15:57, TG3 ≈16:05, TG4 ≈16:13 (AEDT)

2.2 Test plan structure

The plan (`jmeterplan-35.jmx`) contains HTTP Request Defaults, an HTTP Cookie Manager (to keep the session), a **setUp Thread Group** and four load Thread Groups. Because the ParaBank database is reset regularly, the setUp group creates its test data at runtime instead of using hard-coded IDs: it registers a new user with a unique name (`grp35_${__time(,)}`) and opens a second account, which is needed for Transfer Funds. Extractors capture the customer ID and account IDs for later requests. All virtual users share this one account.

#	Request
01	Home Page (after 300 ms Constant Timer)
02	Login (POST `/parabank/login.htm`)
03 / 03b	Accounts Overview + accounts XHR
04 / 04b	Account Details + account details XHR
05 / 05b	Transfer Funds page + transfer XHR
06	View Transactions (XHR)
07	Logout

Every request has at least one assertion, and different assertion types are used across the plan (Response text/code/headers/substring, JSON, Size and Duration). For example, the Login assertion checks that the Response Headers contain `overview.htm`, which proves the login redirected to the account overview (Section 3).

2.3 Workload

Thread Group	Users	Ramp-up	Duration	Loops	On sampler error
TG1	1	10 s	20 s	Infinite	Continue
TG2	10	10 s	20 s	Infinite	Continue
TG3	20	10 s	20 s	Infinite	Continue
TG4	50	10 s	20 s	Infinite	Continue

Run procedure. "Run Thread Groups consecutively" is ticked in the plan. For the final results, each load group was run on its own (with setUp enabled every time) and with a gap of at least ~7 minutes between runs (from the response `Date` headers: TG2 → TG3 ≈ 8.5 min, TG3 → TG4 ≈ 7.5 min; TG1 → TG2 was longer). We did this because Cloudflare's 429 responses carry `Retry-After: ~285 s` (Section 5), so running the groups back to back would make each later group start while the previous block was still active. Each group has its own Aggregate Report and View Results Tree listener.

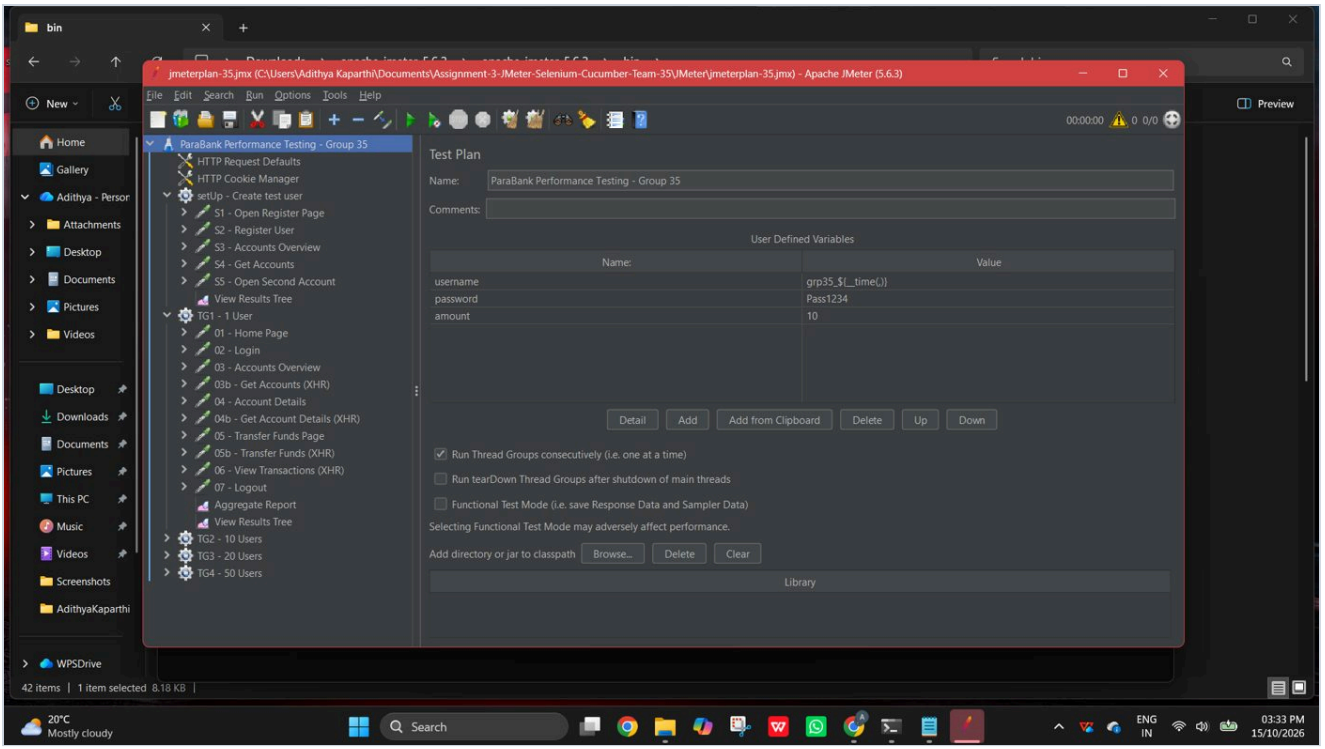


Figure 1. Test plan tree, user-defined variables, and "Run Thread Groups consecutively".

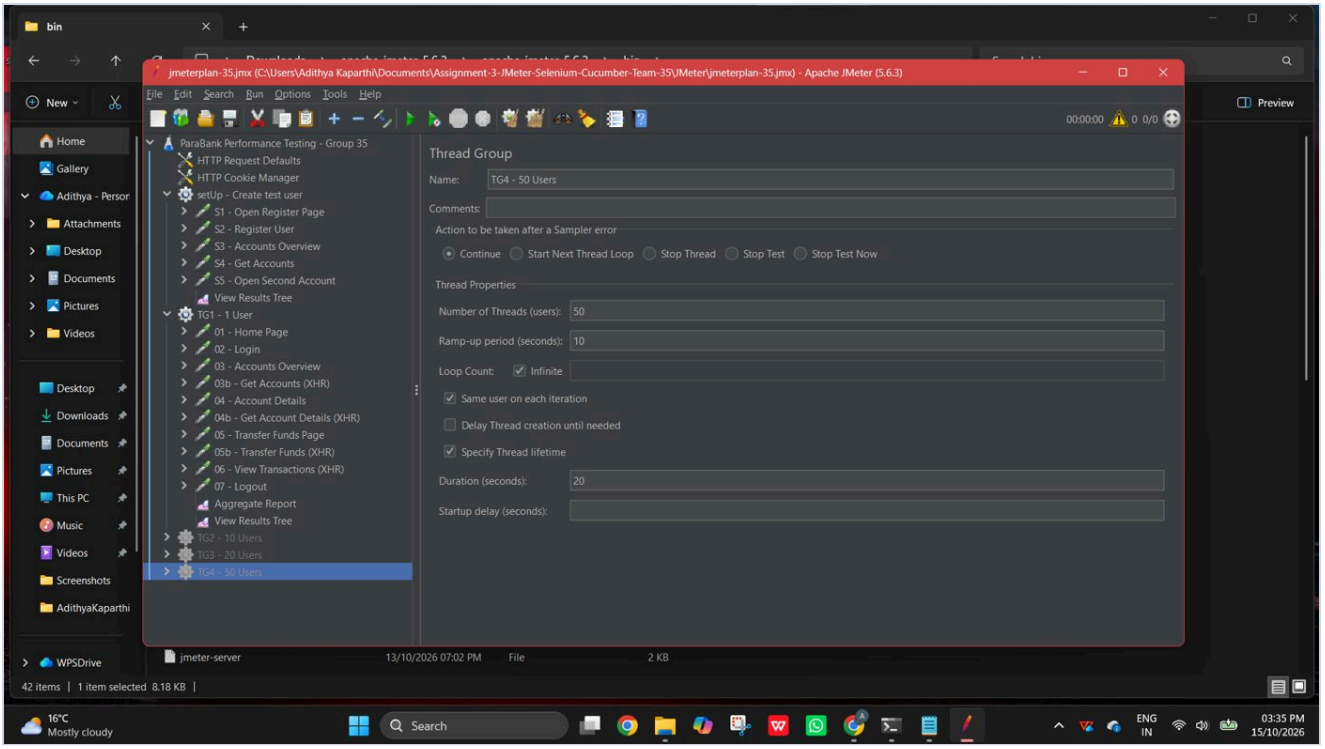


Figure 2. Thread Group settings (TG4 shown: 50 users, 10 s ramp-up, infinite loop, 20 s duration, Continue on error).

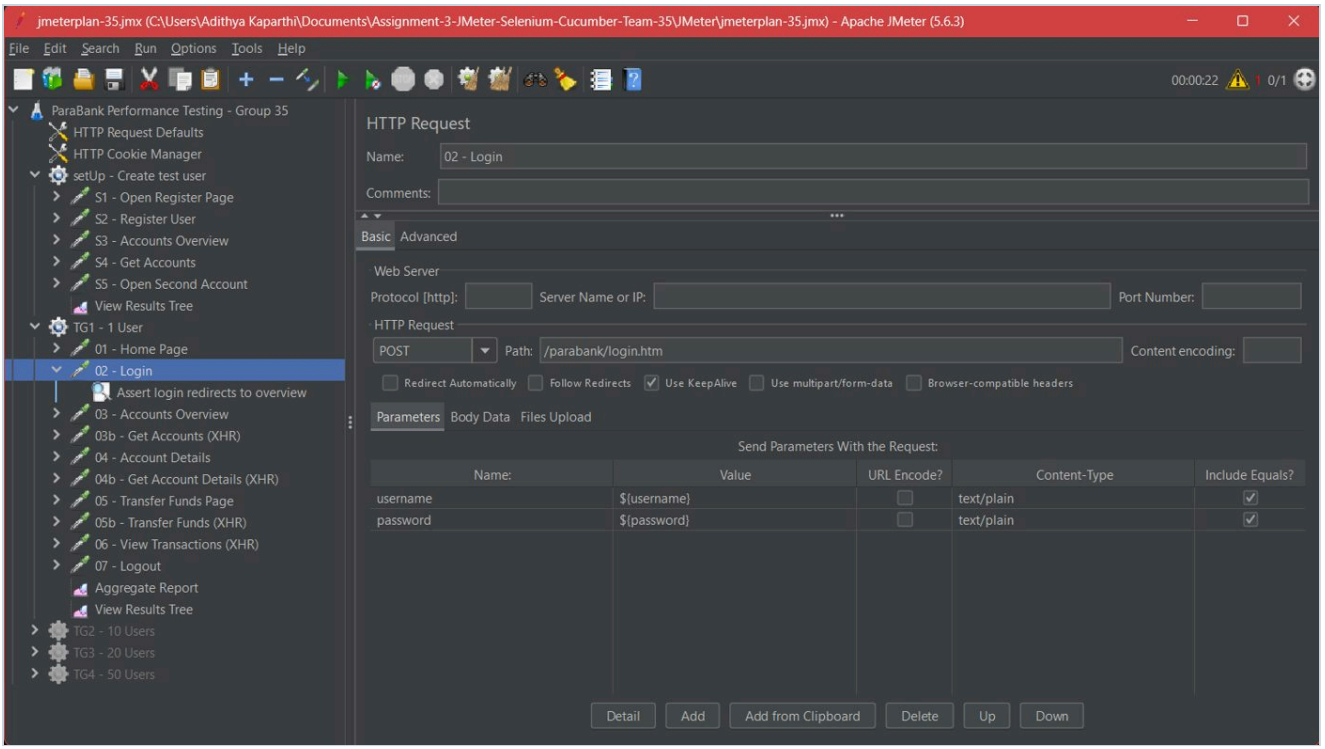


Figure 3. 02 - Login request: POST to `/parabank/login.htm` using the generated `${username}` / `${password}`.

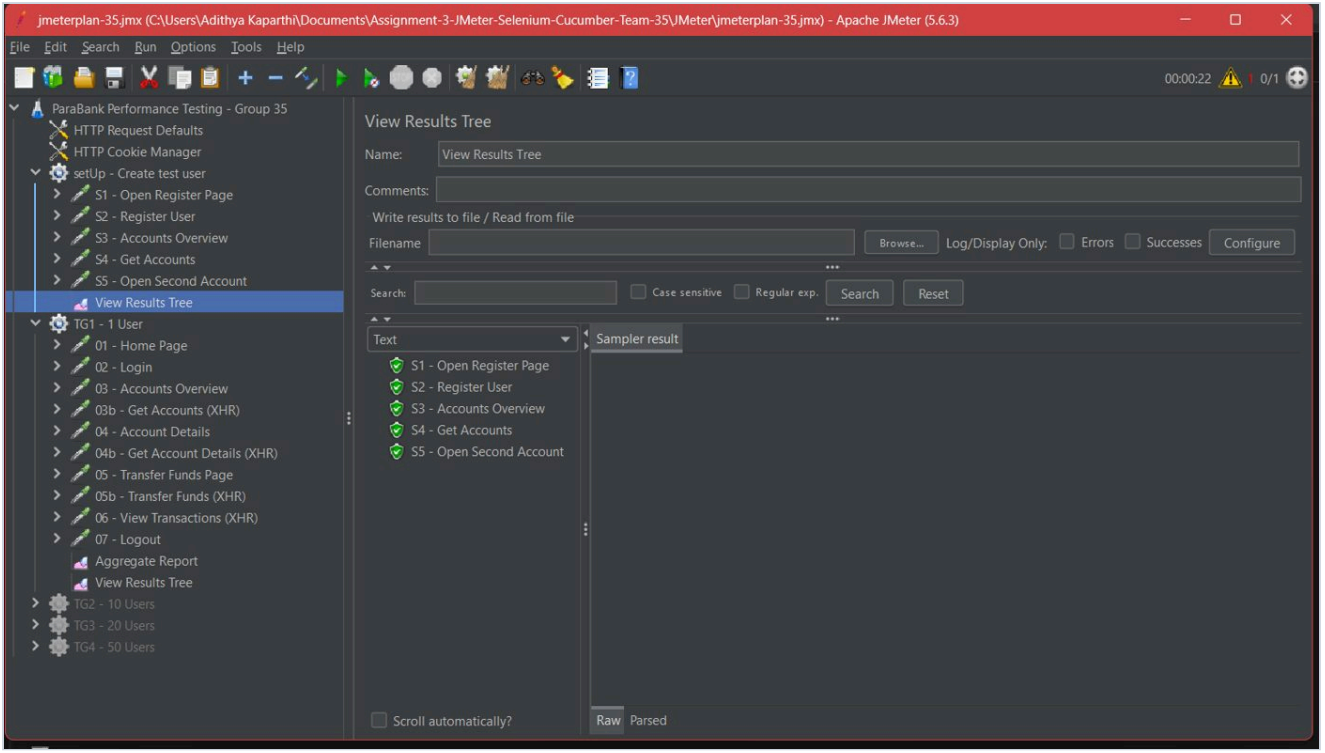


Figure 4. setUp group: S1–S5 all pass (user registered, second account opened). setUp also passed before the 10- and 50-user runs (Appendix A1, A2).

3. Assertion verification

To confirm the assertions really detect bad responses, the Login assertion (Response Headers contain `overview.htm`) was changed to `XYZ123` and TG1 was run again. Only **02 - Login** failed; all other requests stayed green. The server still returned `302 Found → overview.htm`, so the failure came from the assertion, not from the site. The assertion was then restored to `overview.htm`.

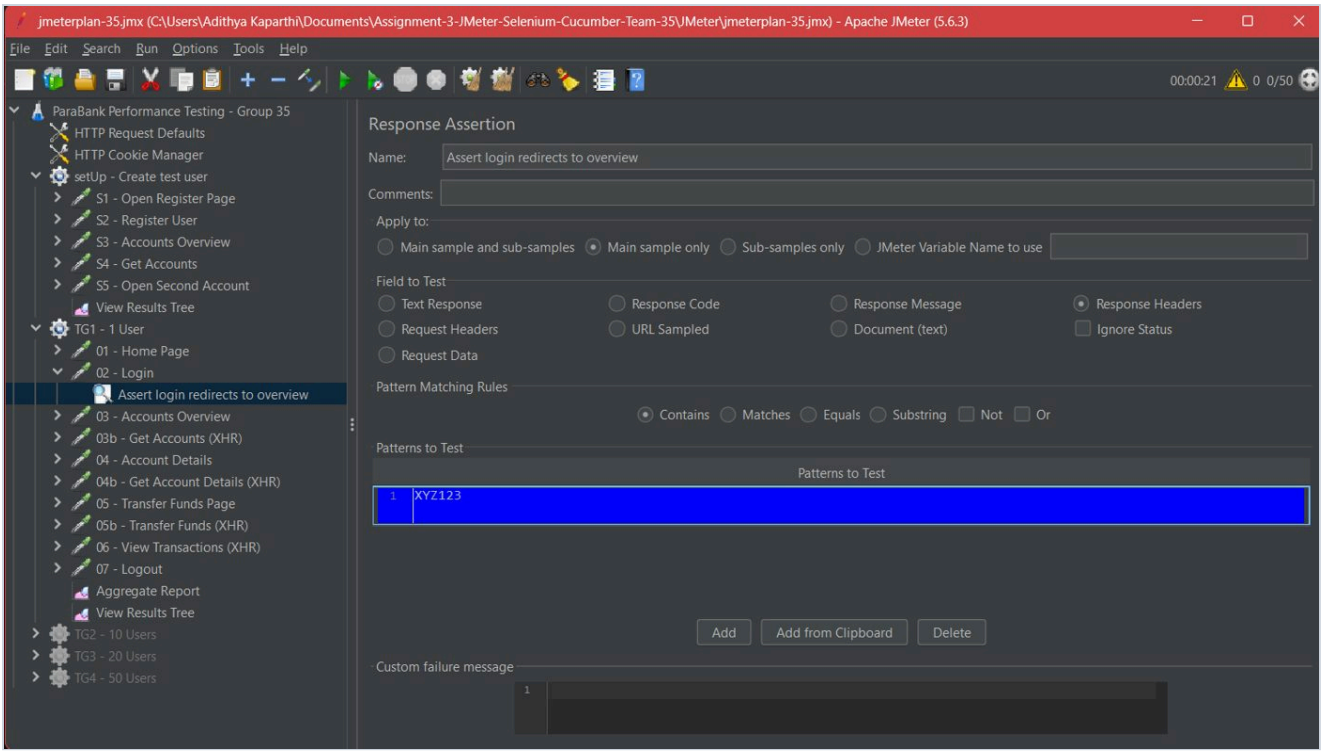


Figure 5. Assertion edited to the wrong value `XYZ123`.

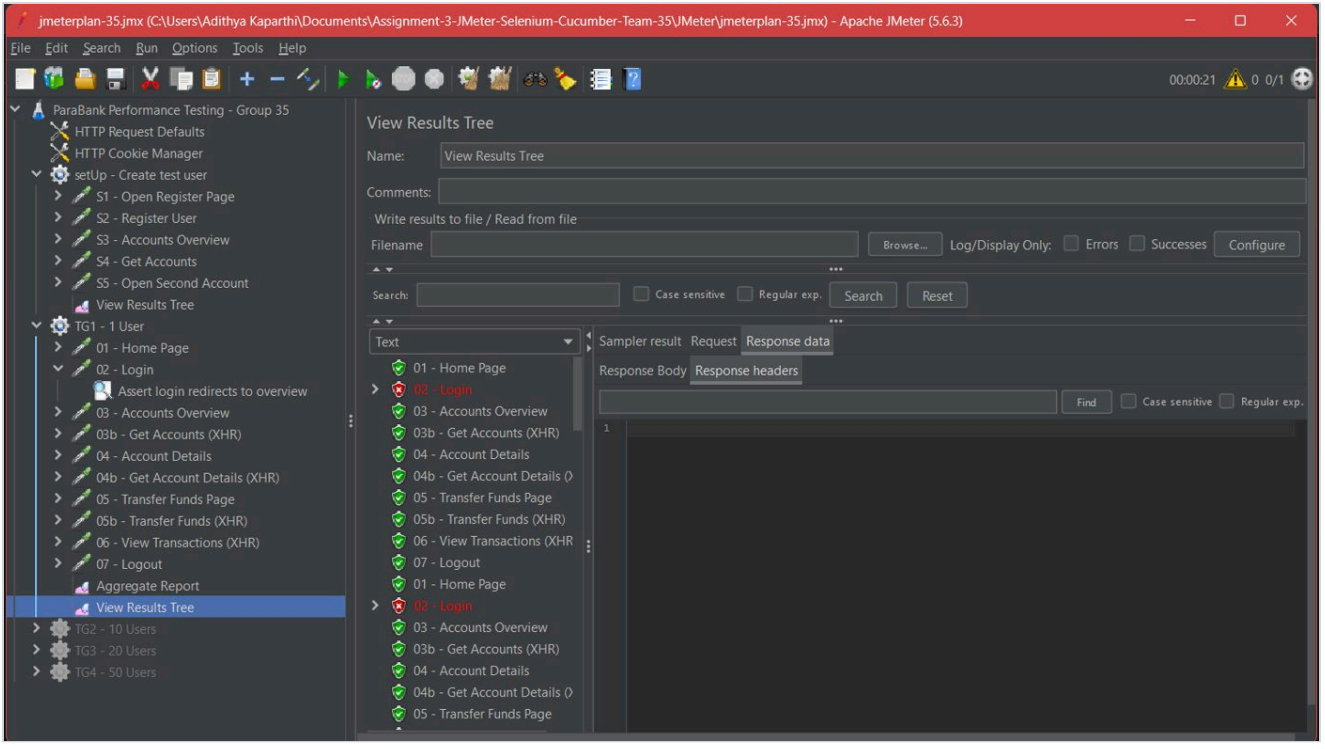


Figure 6. Result: only 02 - Login is marked failed.

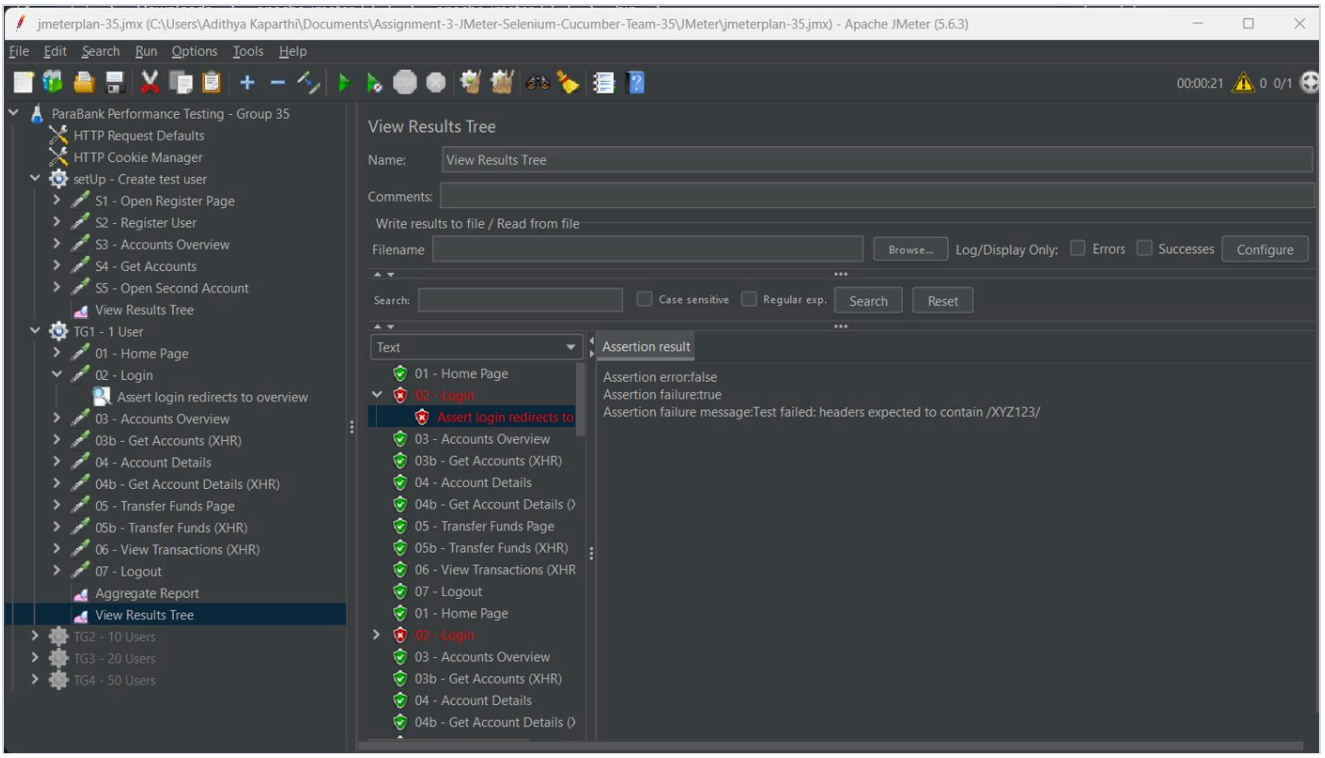
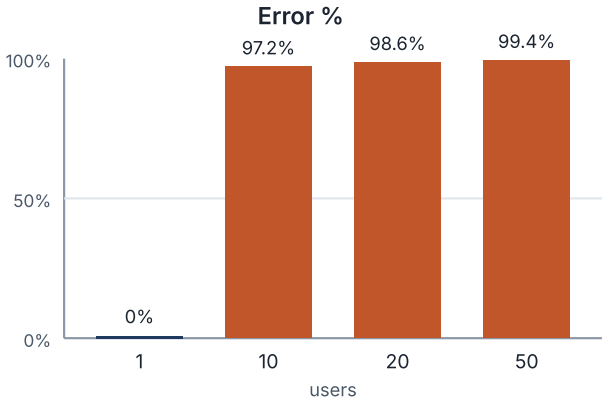


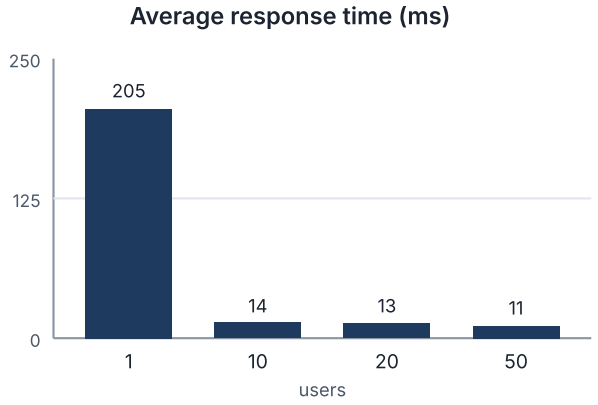
Figure 7. Failure message: "Test failed: headers expected to contain /XYZ123/".

4. Results

4.1 Summary (Aggregate Report TOTAL rows)

Metric	1 user	10 users	20 users	50 users
Samples	84	3,156	6,742	16,712
Throughput	**4.2 /s**	**160.1 /s**	**342.0 /s**	**842.6 /s**
Average	**205 ms**	**14 ms**	**13 ms**	**11 ms**
Min	**192 ms**	**5 ms**	**5 ms**	**5 ms**
Max	**270 ms**	**345 ms**	**630 ms**	**616 ms**
95th percentile	**227 ms**	**19 ms**	**20 ms**	**14 ms**
Error %	**0.00%**	**97.18%**	**98.55%**	**99.42%**
Median	202 ms	9 ms	9 ms	9 ms
99th percentile	253 ms	197 ms	202 ms	45 ms
Successful samples (≈ Samples × (1 − Error %))	84	≈ 89	≈ 98	≈ 97





4.2 Error % per request (first and last in the scenario)

Load	01 - Home Page	07 - Logout
1 user	0.00%	0.00%
10 users	96.21%	98.10%
20 users	98.08%	98.96%
50 users	99.05%	99.76%

JMeter lists Aggregate Report rows in the order each request first runs, which matches the plan order (01 → 07). The sample counts confirm this: they stay the same or fall by one down the list, because the 20-second run can stop part-way through an iteration.

4.3 1 user (TG1)

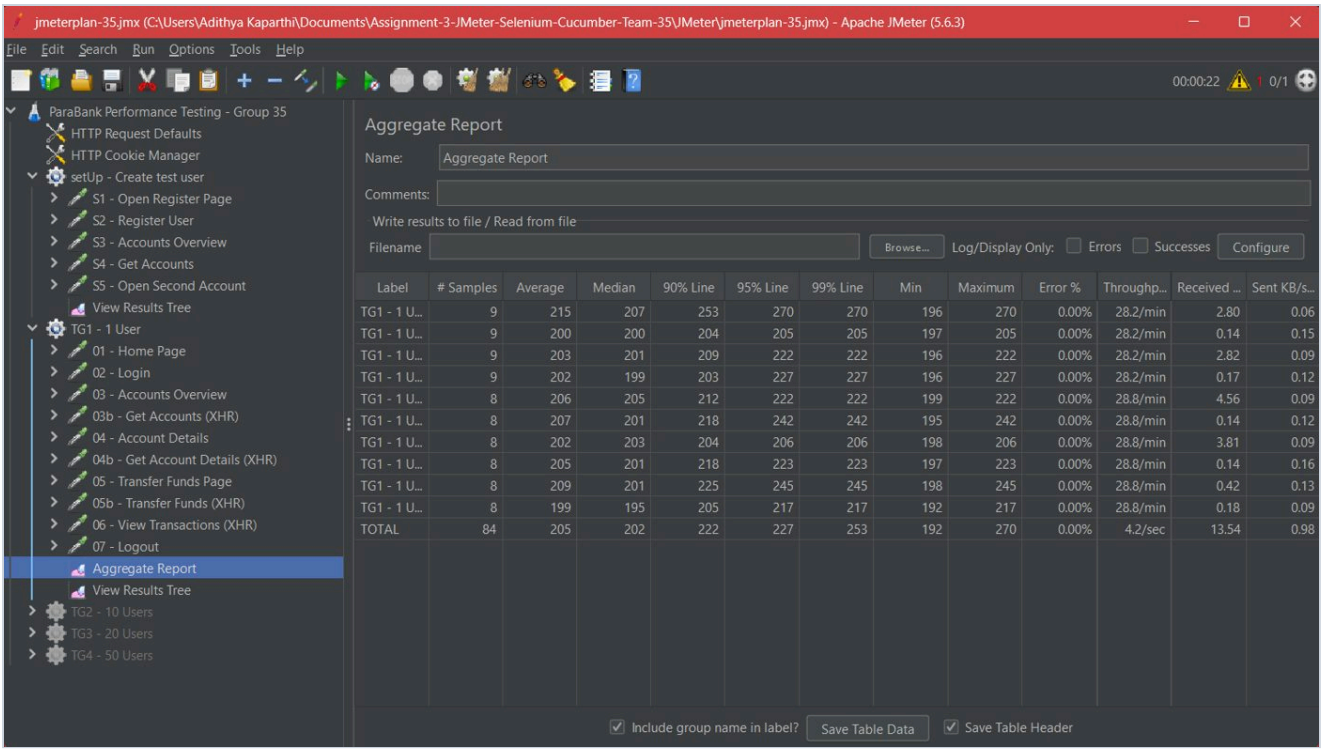


Figure 8. TG1 Aggregate Report.

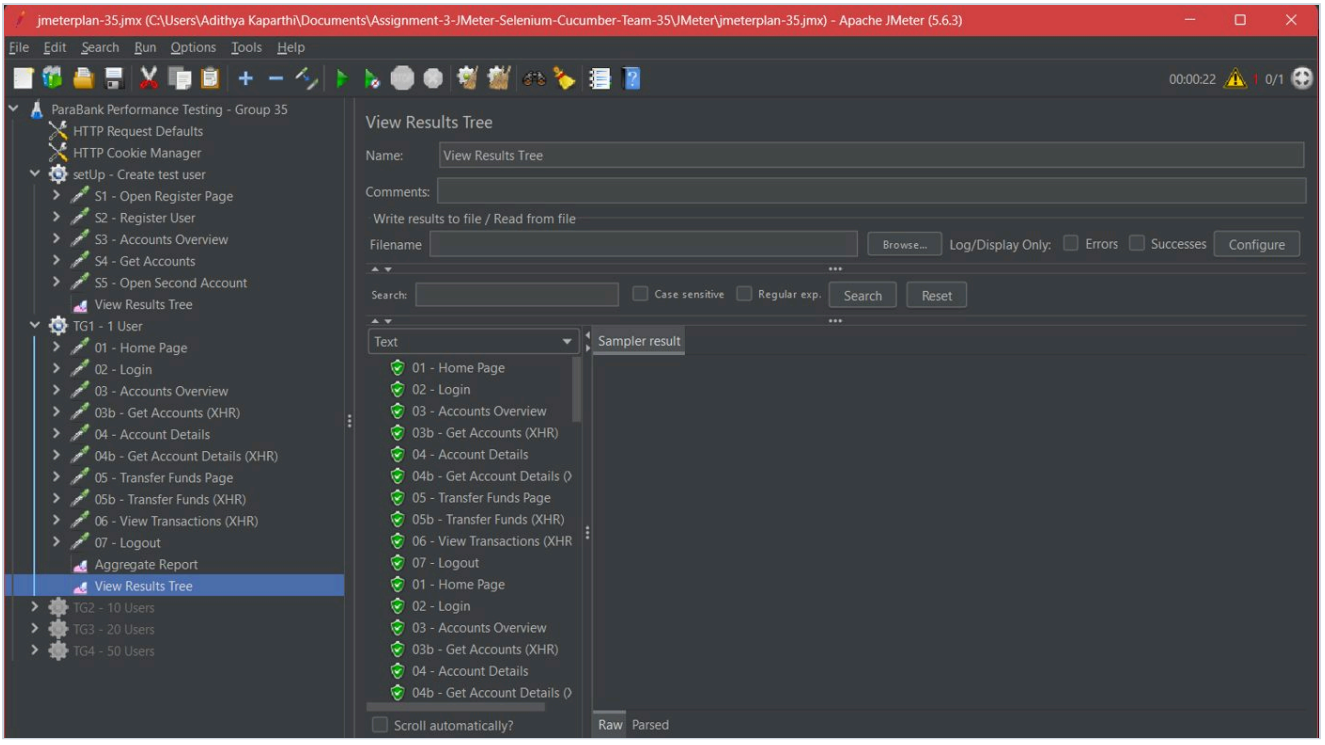


Figure 9. TG1 View Results Tree: all requests pass.

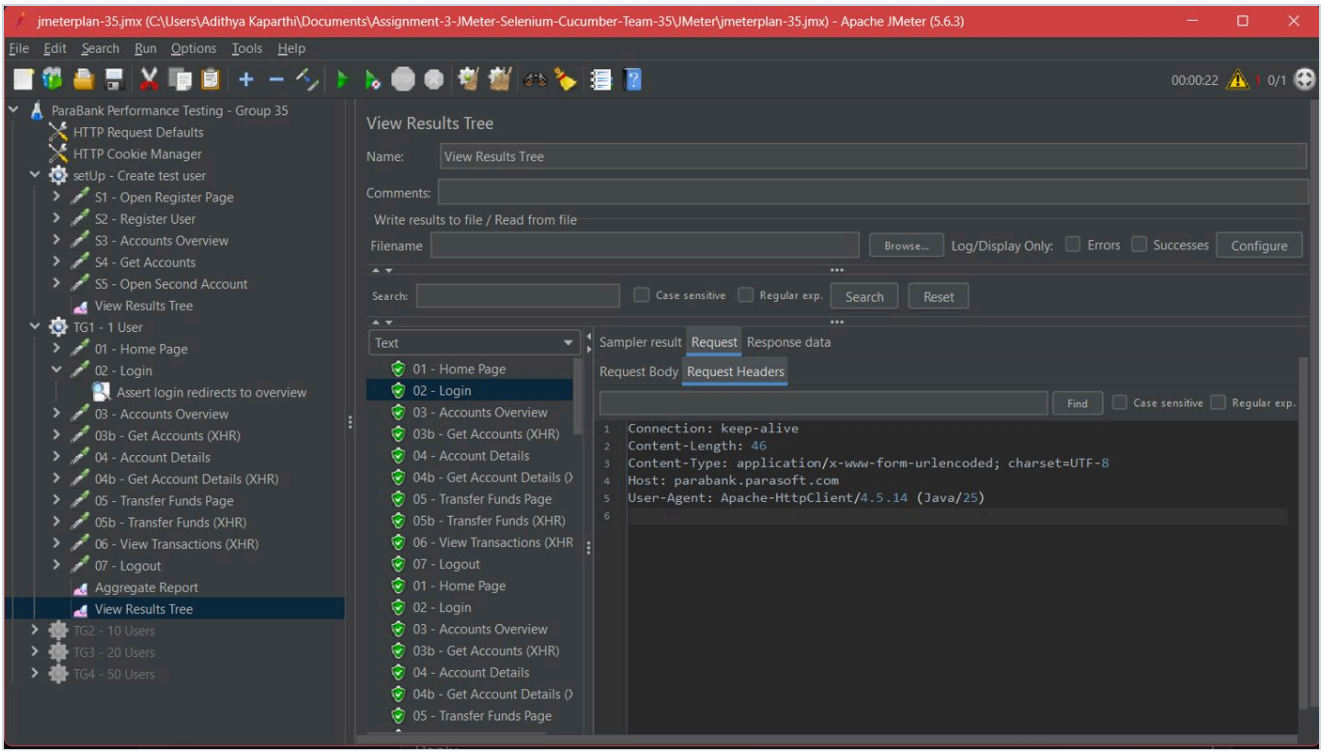


Figure 10. TG1, 02 - Login: request headers.

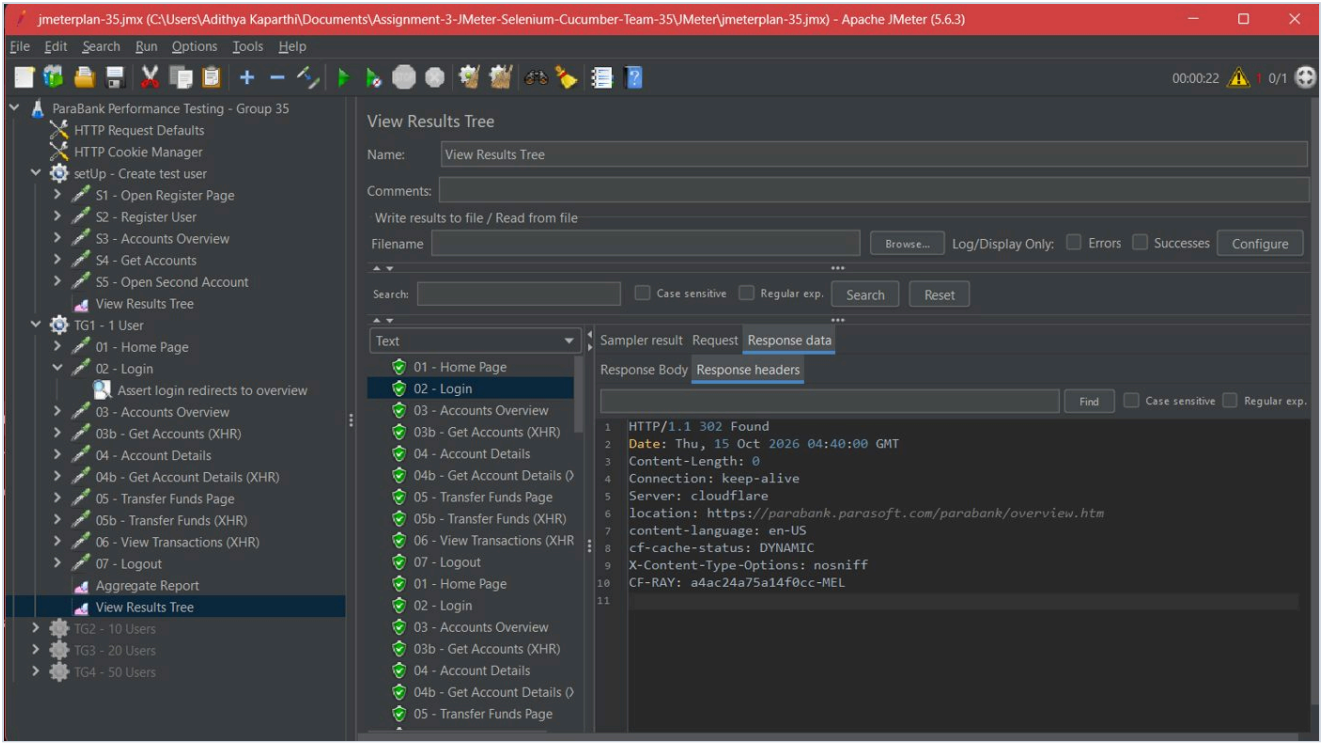


Figure 11. TG1, 02 - Login: response headers (`302 Found`, `location: …/overview.htm`, `Server: cloudflare`).

4.4 10 users (TG2)

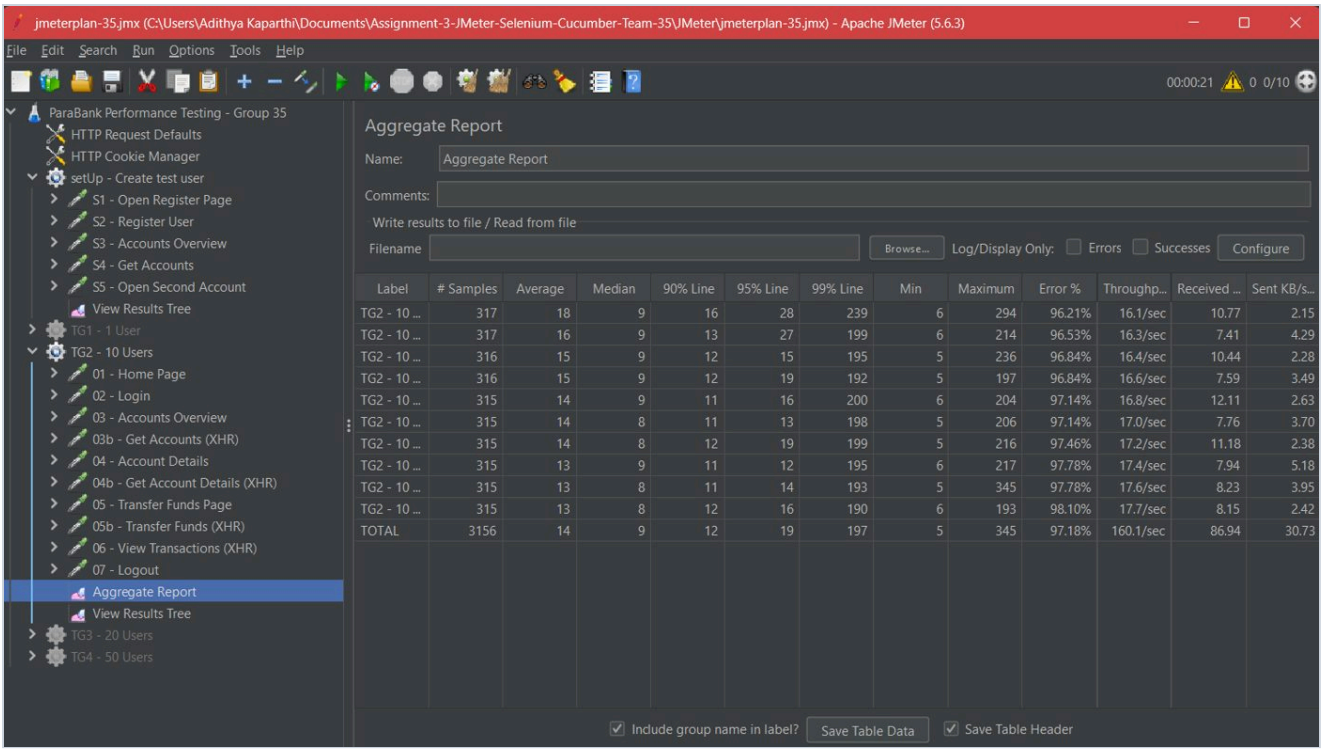


Figure 12. TG2 Aggregate Report.

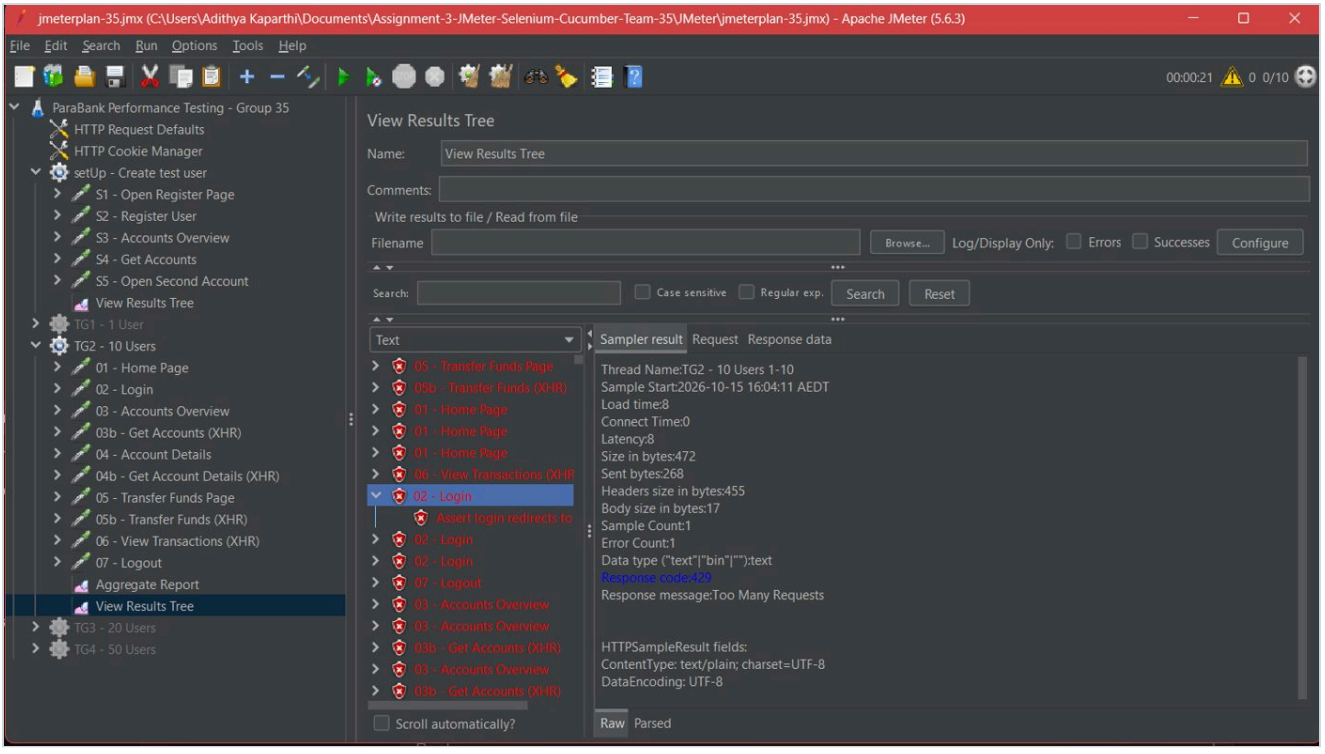


Figure 13. TG2, 02 - Login sampler result: `429 Too Many Requests`, 8 ms, 17-byte body.

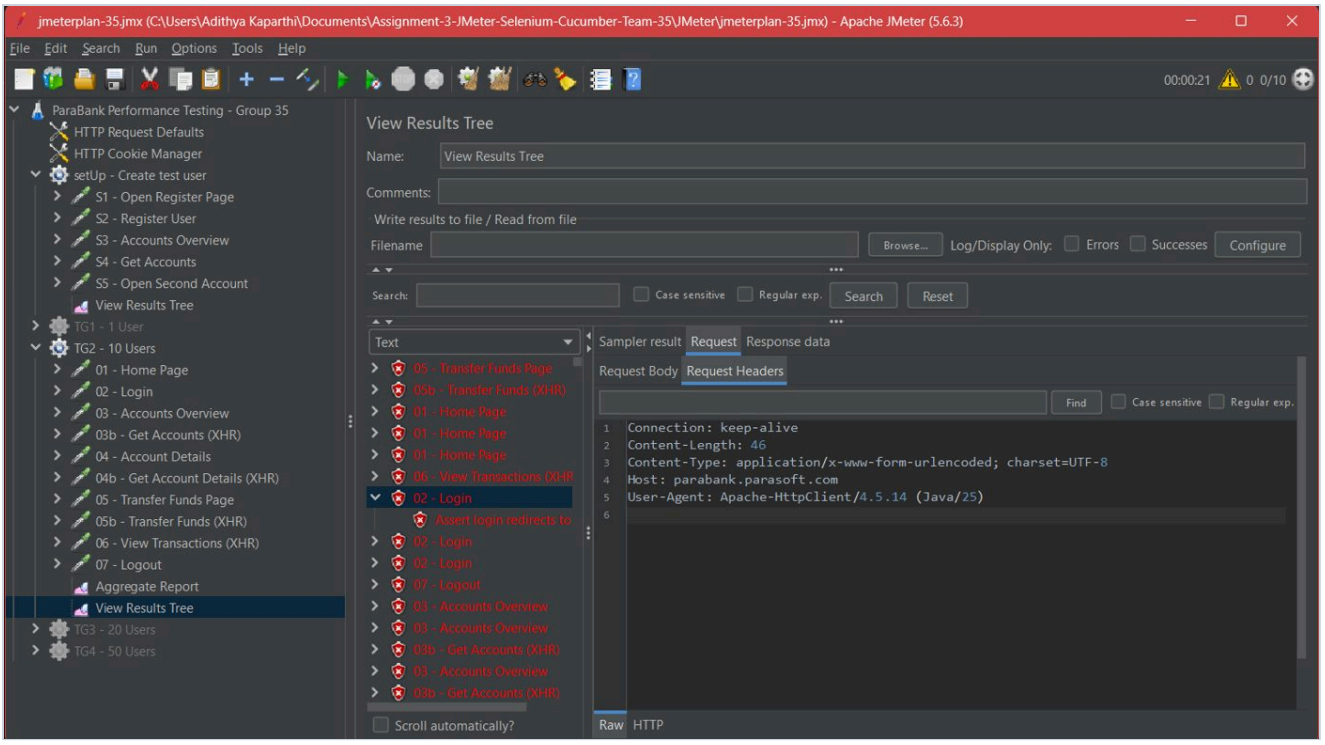


Figure 14. TG2, 02 - Login: request headers (identical to TG1).

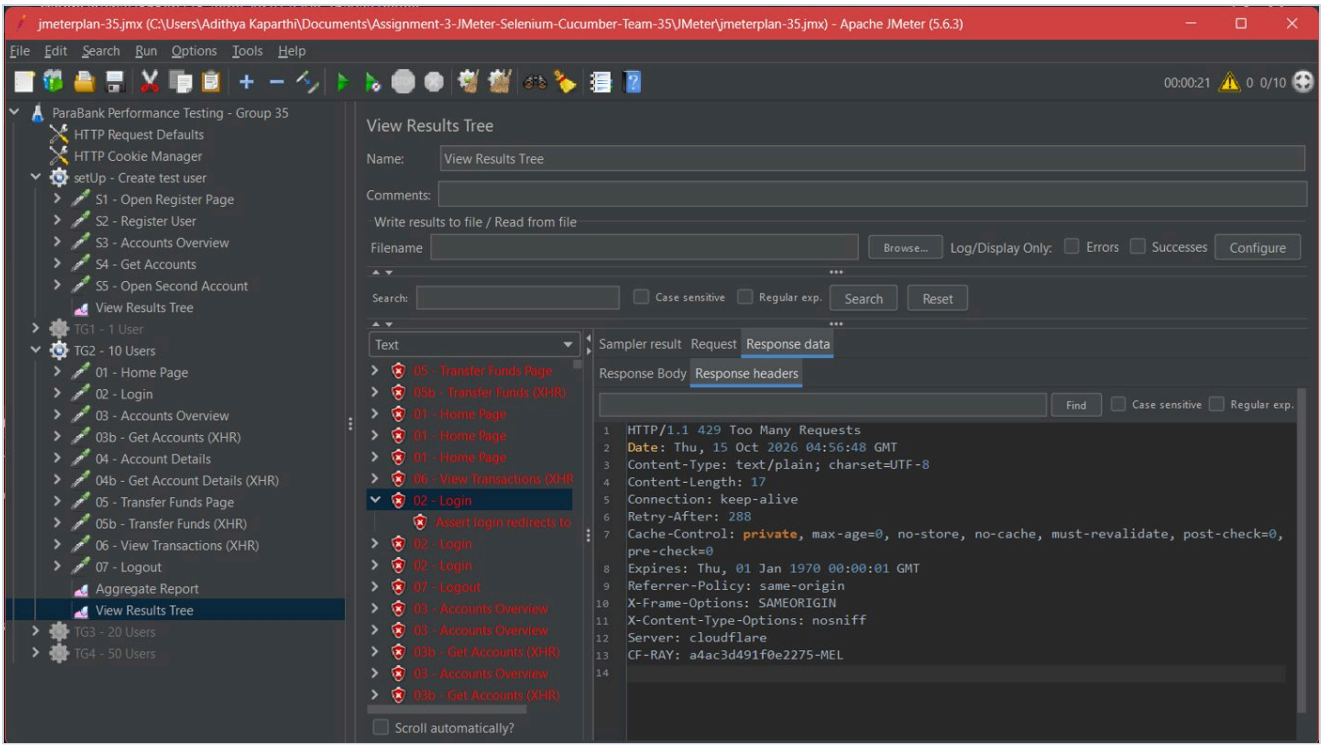


Figure 15. TG2, 02 - Login: response headers (`429`, `Retry-After: 288`, `Server: cloudflare`).

4.5 20 users (TG3)

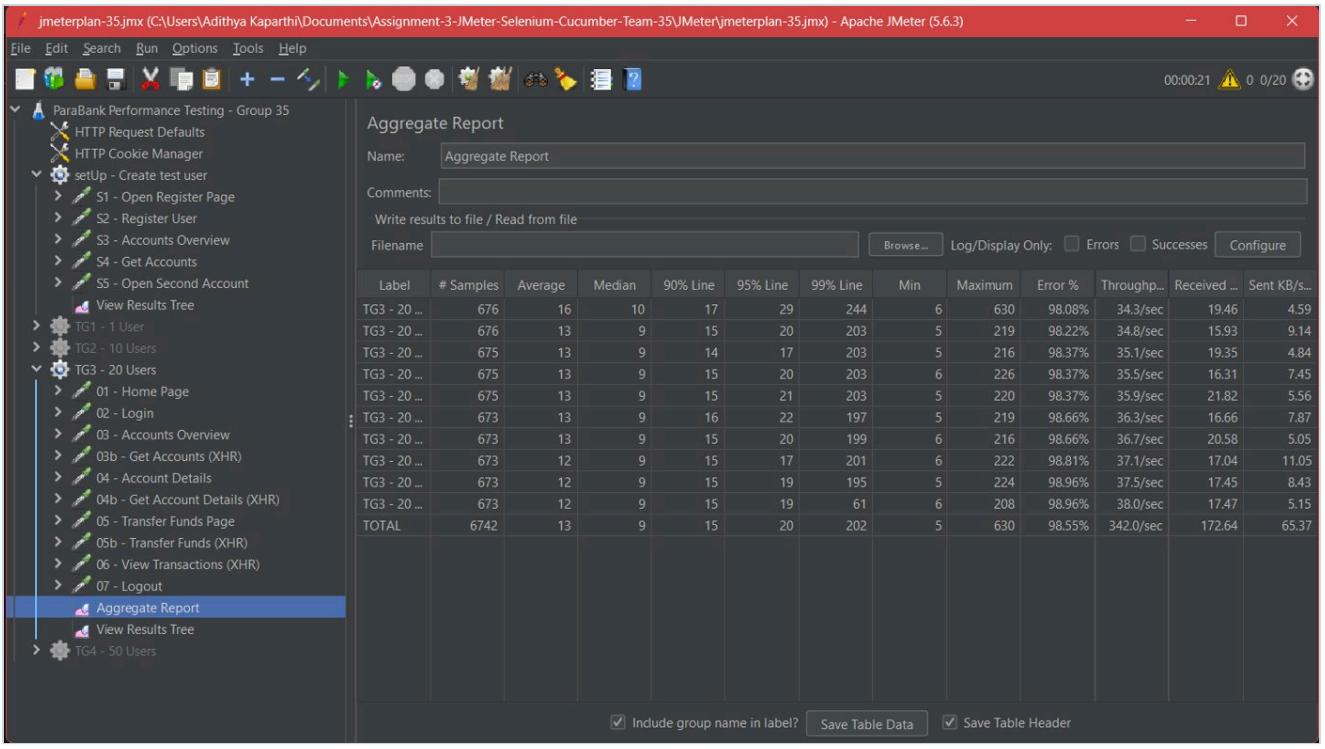


Figure 16. TG3 Aggregate Report.

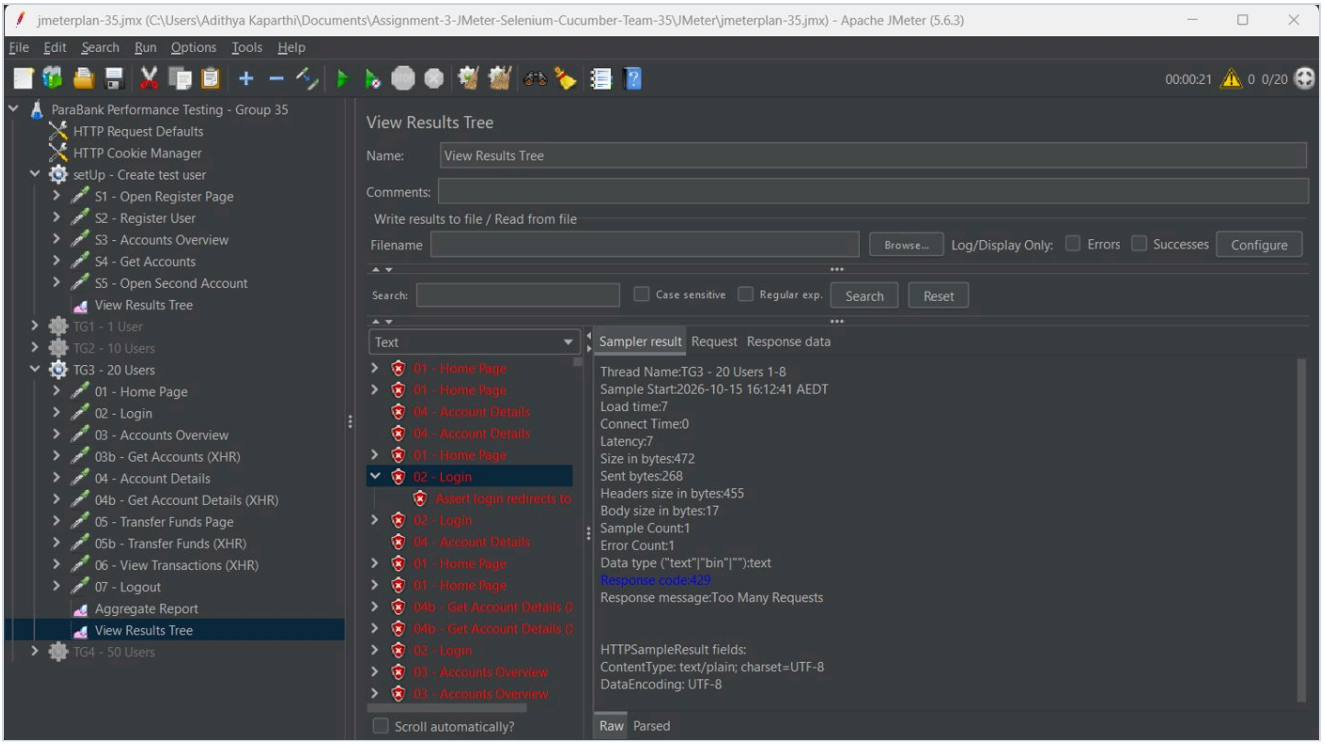


Figure 17. TG3, 02 - Login sampler result: `429`, 7 ms.

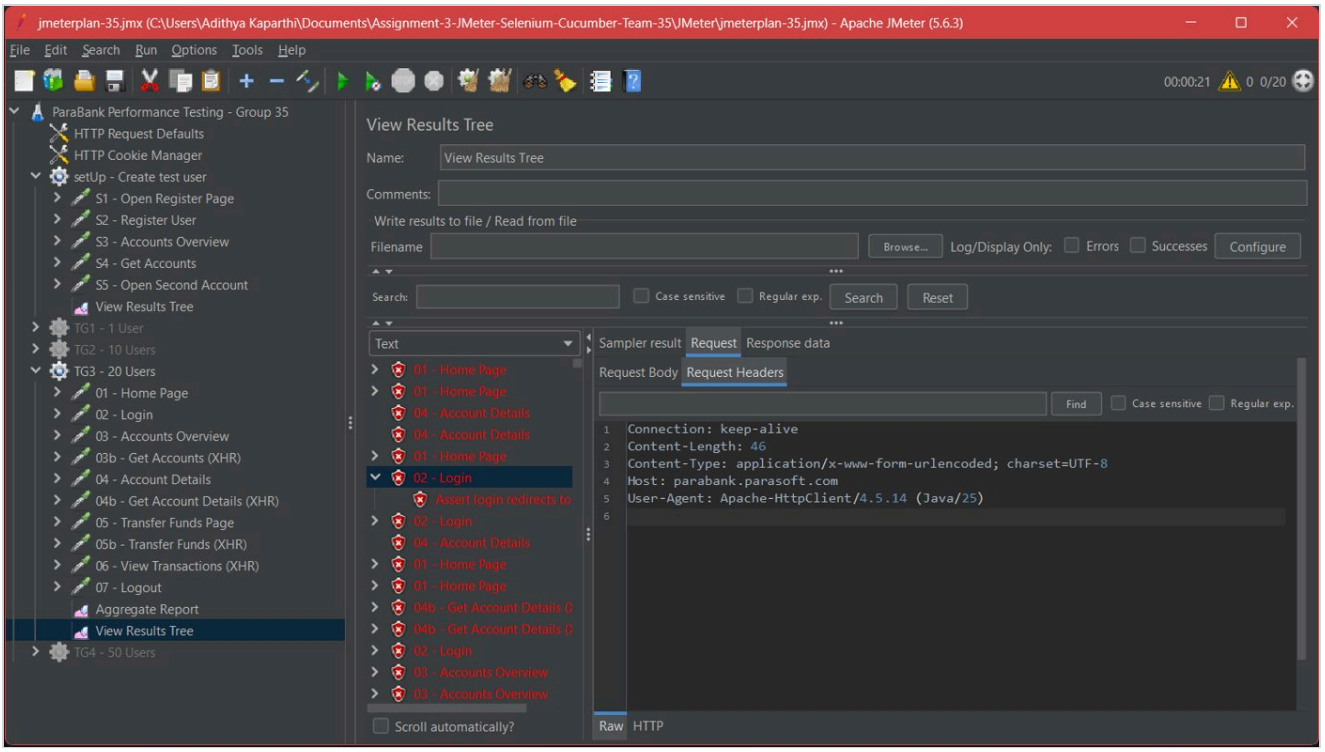


Figure 18. TG3, 02 - Login: request headers.

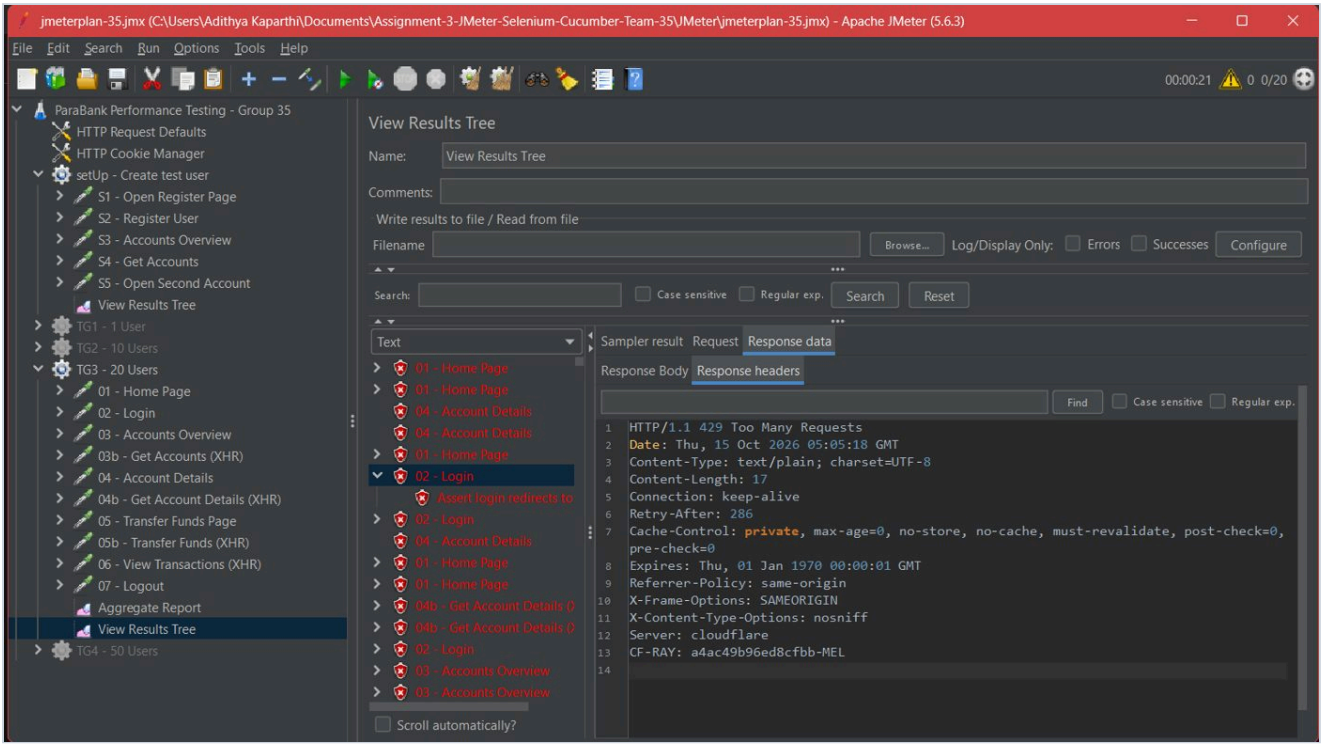


Figure 19. TG3, 02 - Login: response headers (`Retry-After: 286`).

4.6 50 users (TG4)

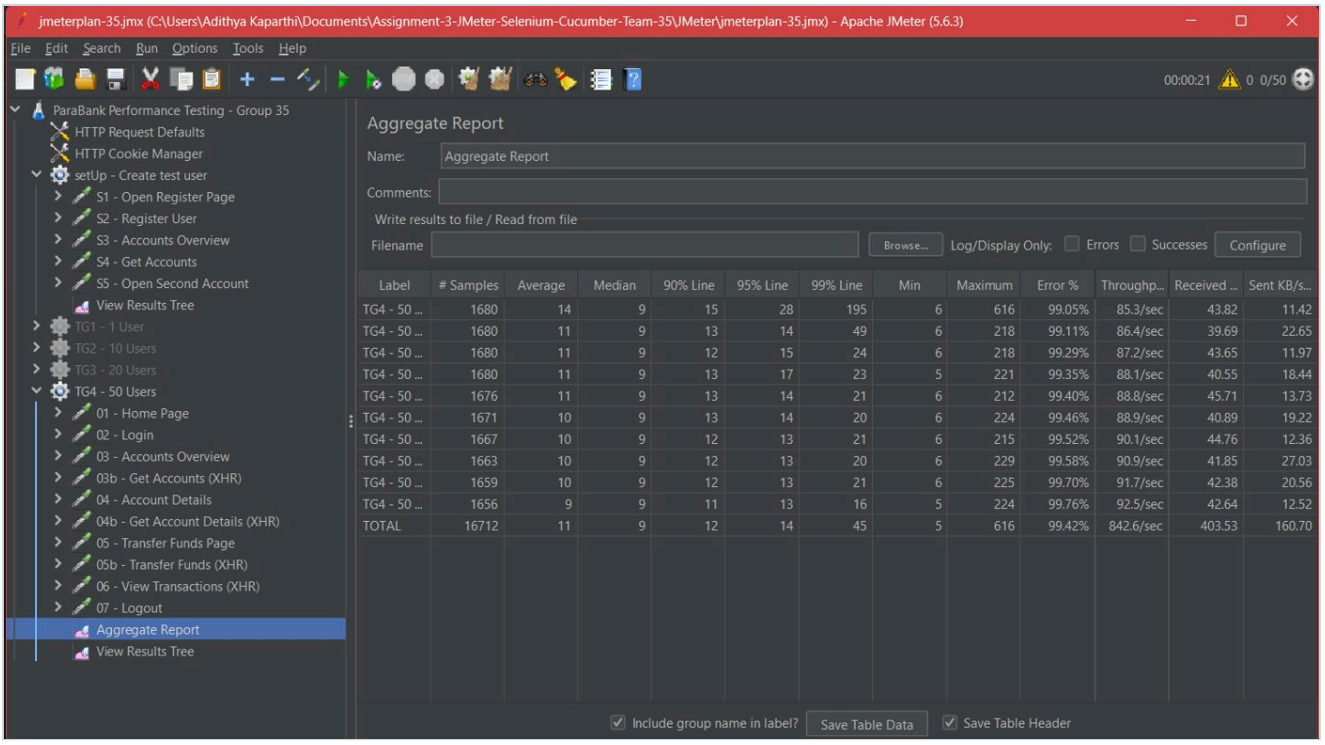


Figure 20. TG4 Aggregate Report.

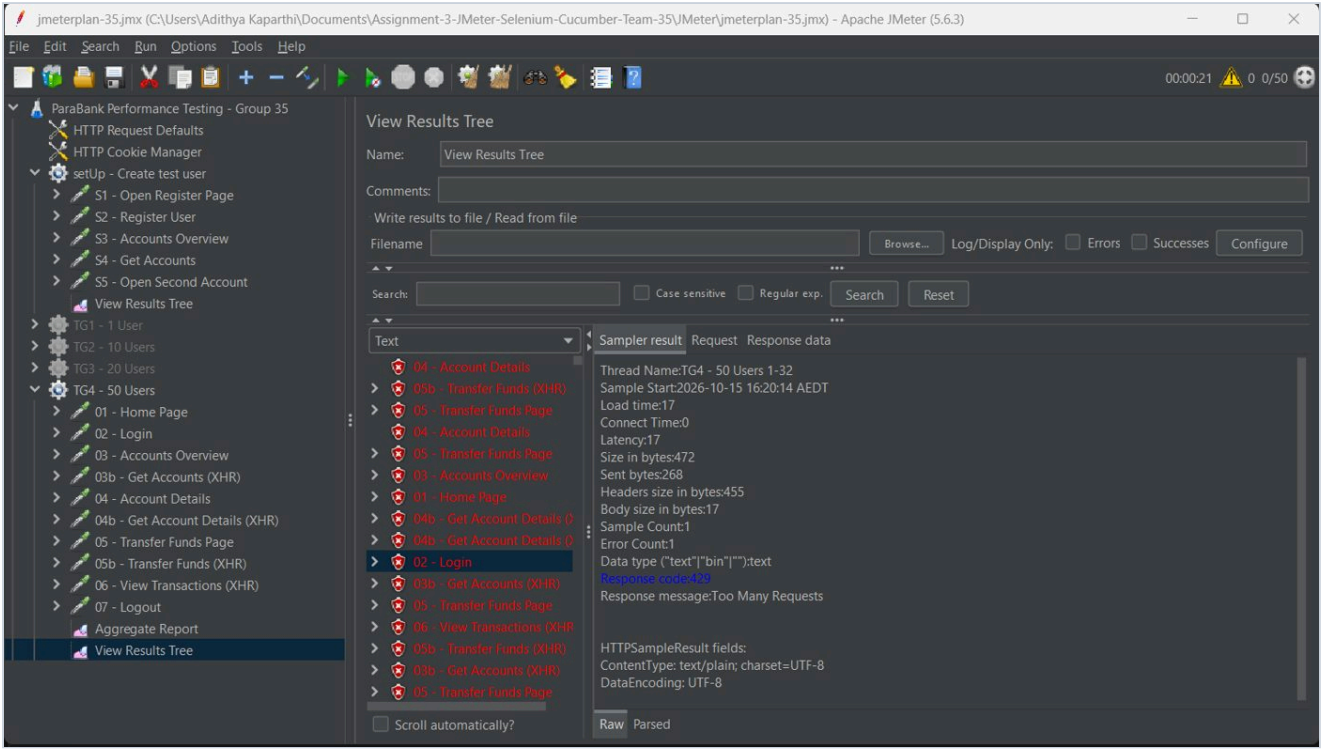


Figure 21. TG4, 02 - Login sampler result: `429`, 17 ms.

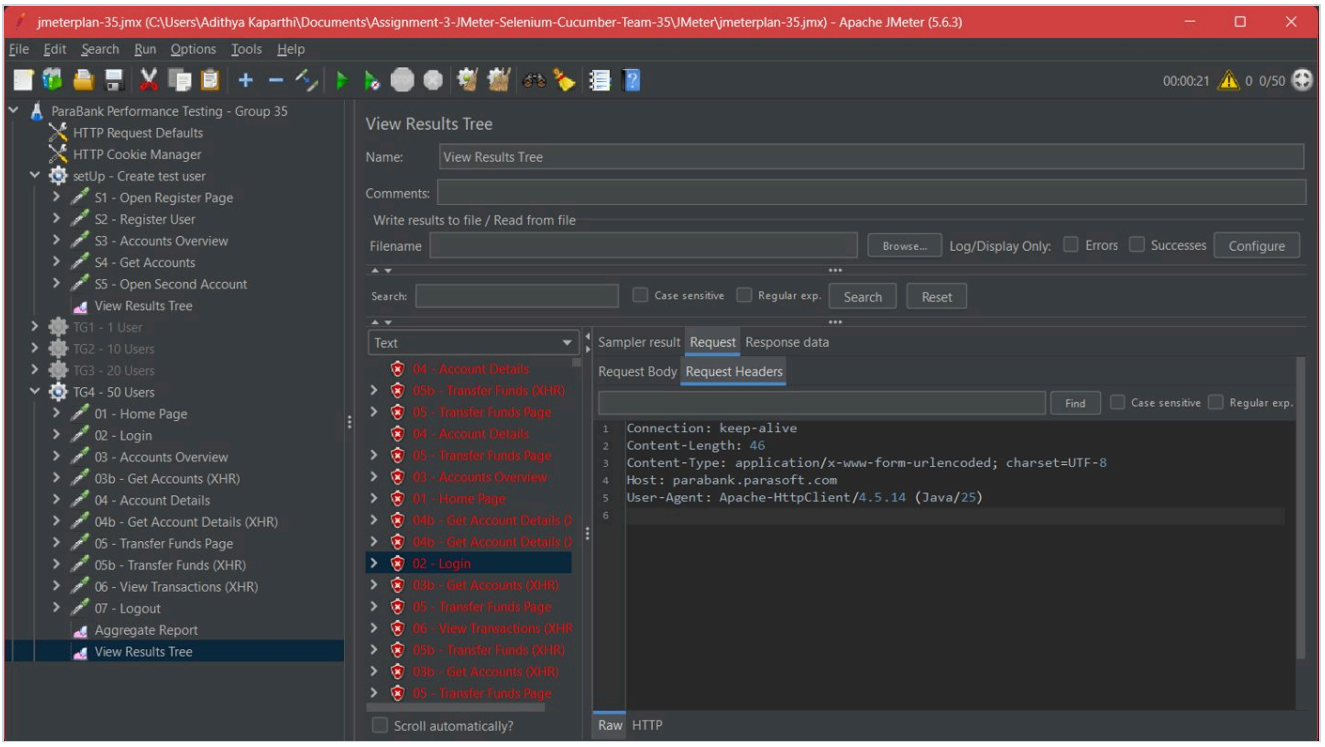


Figure 22. TG4, 02 - Login: request headers.

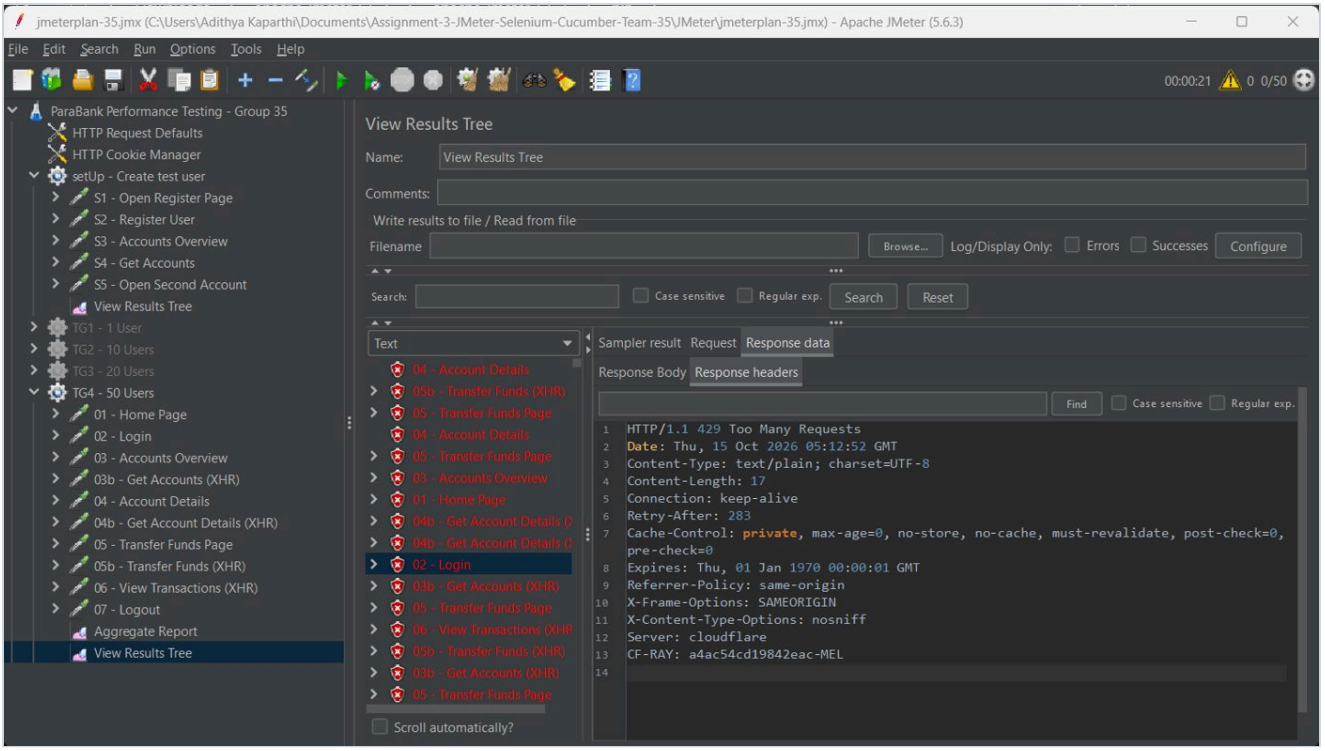


Figure 23. TG4, 02 - Login: response headers (`Retry-After: 283`).

5. Analysis

5.1 Baseline: 1 user

With one user, every request passed (0% errors). Response times were very stable: Min 192 ms, Average 205 ms, 95th percentile 227 ms and Max 270 ms, a spread of under 80 ms. Throughput was 4.2 requests/s. Because the 300 ms timer and the server time are the only waits, this is the normal speed of ParaBank from our location. Most of the ~200 ms is likely network round-trip time to the server rather than processing time, since even very small responses such as the Login redirect (0-byte body) took about 200 ms.

5.2 From 10 users: rate limiting by Cloudflare

At 10, 20 and 50 users the Error % jumped to 97–99%. The View Results Tree shows why: the failing requests got **HTTP 429 Too Many Requests** from **Cloudflare** (`Server: cloudflare`), with a `Retry-After` header of about 285 seconds. ParaBank itself did not crash. Cloudflare sits in front of the site and blocks a client that sends too many requests in a short time. Because all virtual users come from one laptop (one IP address), Cloudflare sees them as a single client that is sending far too much traffic, and blocks it for about 5 minutes.

The setUp group passed before the 1-, 10- and 50-user runs (Figure 4, A1, A2). No setUp screenshot was taken for the 20-user run, but about 12 of its 676 Login requests succeeded (98.22% errors), which is only possible if the setUp user had been created. The request headers sent at 10–50 users were the same as at 1 user. So the errors were not caused by bad test data or a broken request. Only the server's response changed.

5.3 Why Average and percentiles went down and Throughput went up

At first glance the higher loads look faster: Average fell from 205 ms to 11–14 ms and Throughput rose from 4.2/s to 842.6/s. This is misleading. A 429 rejection is returned by Cloudflare's edge server almost immediately (5–17 ms, 17-byte body) without ever reaching ParaBank. Because 97–99% of samples were these fast rejections, they dominate the Average, Median and 95th percentile. JMeter's Throughput counts failed samples as well, so it measures how fast Cloudflare said "no", not how much real work the site did.

The successful samples tell a different story. Multiplying samples by the success rate gives about 84, 89, 98 and 97 successful requests for 1, 10, 20 and 50 users. In every run, roughly 85–100 real requests got through in 20 seconds, however many users we added. This suggests the useful throughput was capped at about 4–5 requests/s, the same as one user. It fits a rate limit of roughly 90–100 requests per time window from one IP: one user sending 84 requests in 20 s stayed just under it, and 10 users passed it within a few seconds.

5.4 What the Max and 99th percentile show

The few requests that did reach ParaBank got slower as load increased. Max rose from 270 ms (1 user) to 345 ms (10 users) and 630 ms / 616 ms (20 / 50 users). At 10 and 20 users the 99th percentile (≈200 ms) still sits in the range of a real page load, because the top 1–3% of samples are the successful ones. At 50 users only 0.58% of samples succeeded, so even the 99th percentile (45 ms) is a 429 rejection. This means percentile metrics stop describing the real site once errors take up nearly all samples.

5.5 Why Error % rises from Home to Logout

In every load, Error % increases step by step through the scenario (for example 96.21% for 01 Home to 98.10% for 07 Logout at 10 users). Two effects cause this. First, the block starts partway through the first few iterations: the earliest requests (Home, Login) in the first loops still get through, but by the time the later requests are sent, the limit has already been reached. Second, the steps depend on each other through the shared session cookie. If Login is rejected, the user has no valid session, so later requests such as Account Details or Transfer Funds fail even when they are not rate-limited themselves.

5.6 Other factors

- **Shared account:** all virtual users log in as the same user and transfer from the same account. On a real system this could cause locking or conflicting balances under load. Here it had little visible effect because so few requests reached the server.
- **Public demo server:** parabank.parasoft.com is shared with other people. Its speed depends on other traffic and on its regular database resets, which we cannot control.
- **Network latency:** the ~200 ms baseline is mostly distance and network delay, so a server closer to the client would show lower response times for the same load.

6. Limitations

- All load came from **one machine and one IP address**, which triggered Cloudflare's rate limit. Real users come from many IP addresses, so these results show the limits of the protection layer more than the capacity of ParaBank itself.
- The runs were done in JMeter **GUI mode**. JMeter recommends non-GUI mode for load testing because the GUI listeners use CPU and memory on the client. At these small loads the client was not the bottleneck, but a non-GUI run (`jmeter -n -t jmeterplan-35.jmx -l results.jtl -e -o report`) would give cleaner numbers.
- Each load was run once for 20 seconds. Results from a single short run on a public server can vary; Praneeth's trial runs on 12–13 Oct showed the same pattern (e.g. 237 ms average at 1 user, 95–99% errors at 10–50 users).
- Successful-only metrics are estimated from the Error % rather than filtered from a raw results file.

7. Conclusion

With one user, ParaBank responded consistently in about 200 ms with no errors. From 10 users upwards, Cloudflare's rate limiting blocked 97–99% of requests with HTTP 429, and the block lasted about 5 minutes. As a result, the raw Average and Throughput figures improve with load, but only because rejected requests are very fast. The number of successful requests stayed at roughly 85–100 per 20-second run at every load, and the slowest successful responses doubled from 270 ms to over 600 ms. The key lesson is that response time and throughput must always be read together with Error %: when the error rate is high, those metrics describe the rejection, not the application.

Appendix – setUp checks and additional evidence

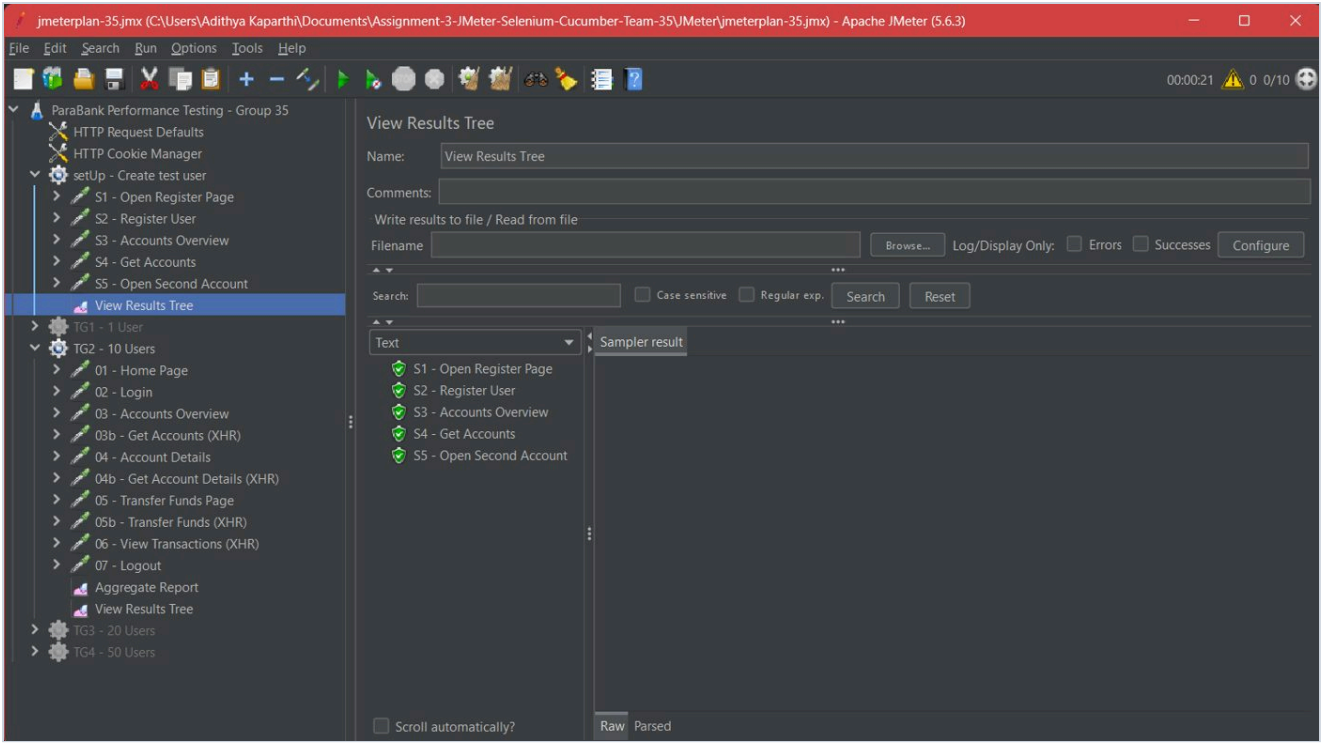


Figure A1. setUp passed before the 10-user run.

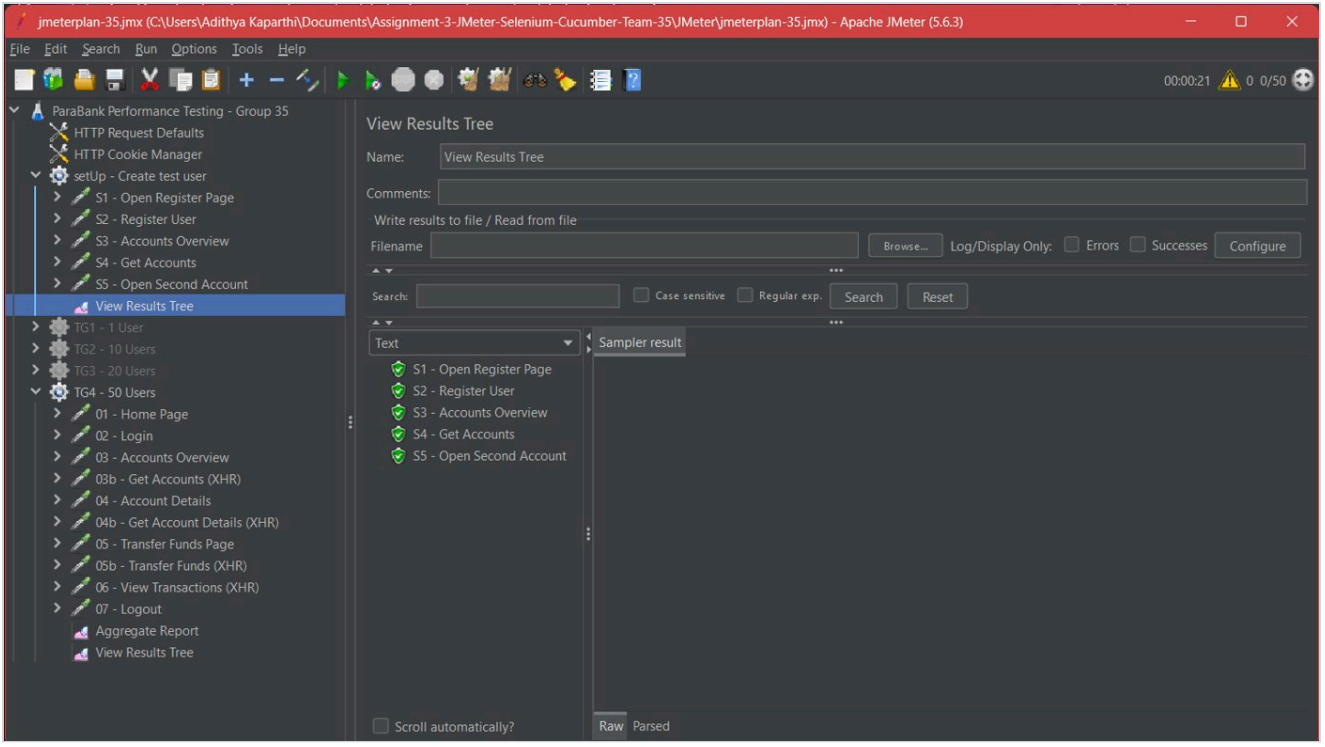


Figure A2. setUp passed before the 50-user run.

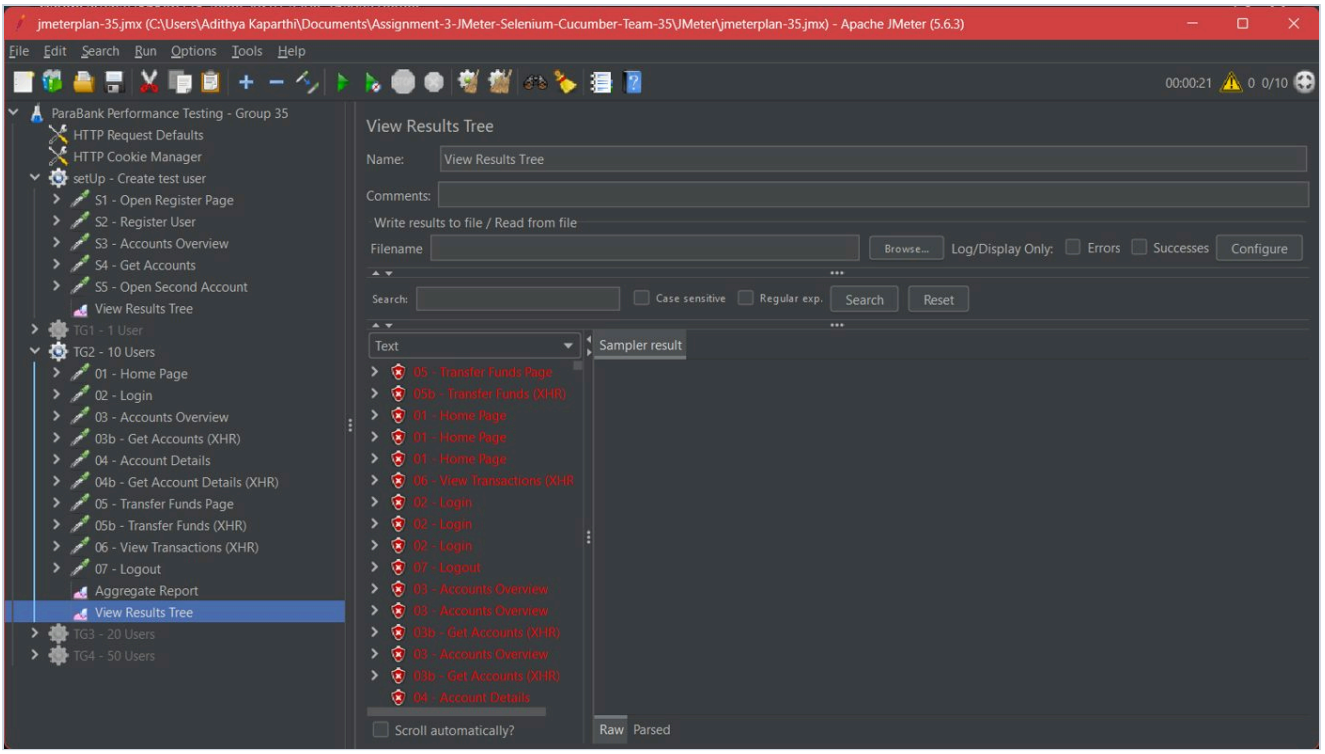


Figure A3. TG2 View Results Tree: almost all samples failed.

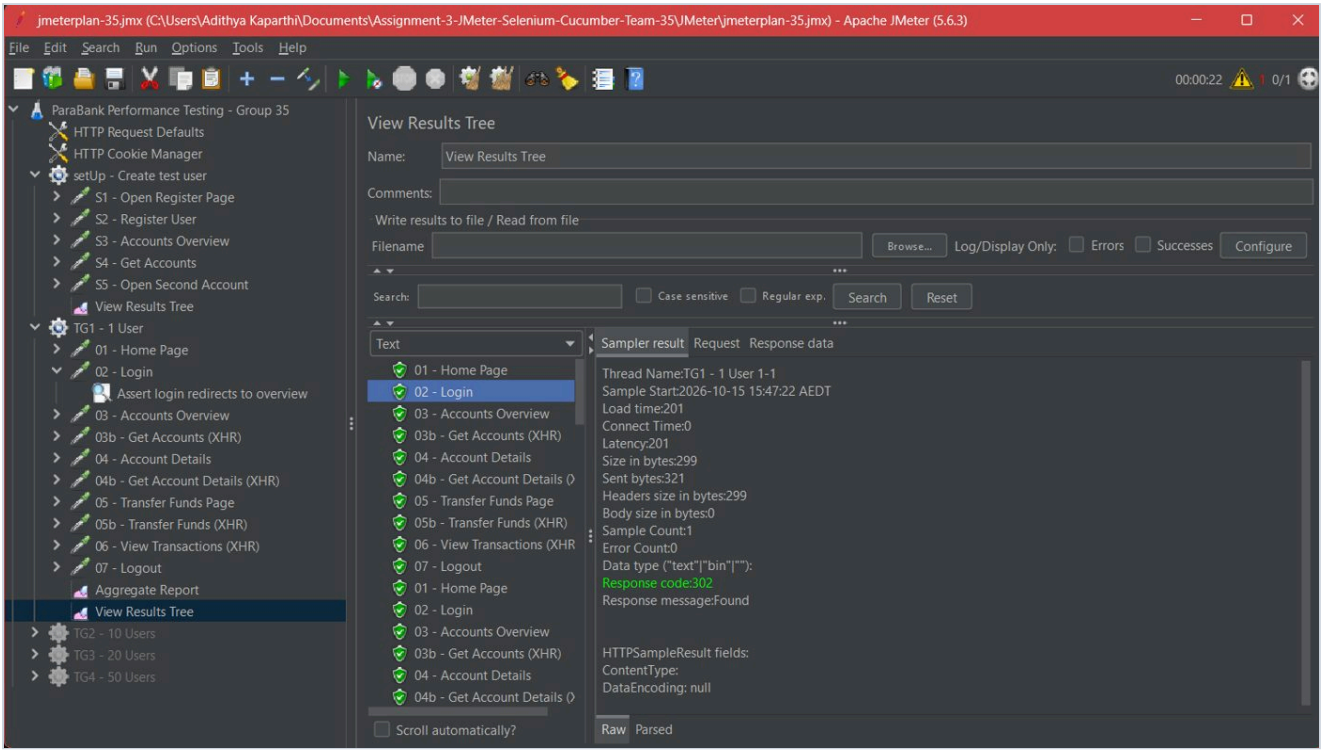


Figure A4. TG1, 02 - Login sampler result: `302 Found` in 201 ms.

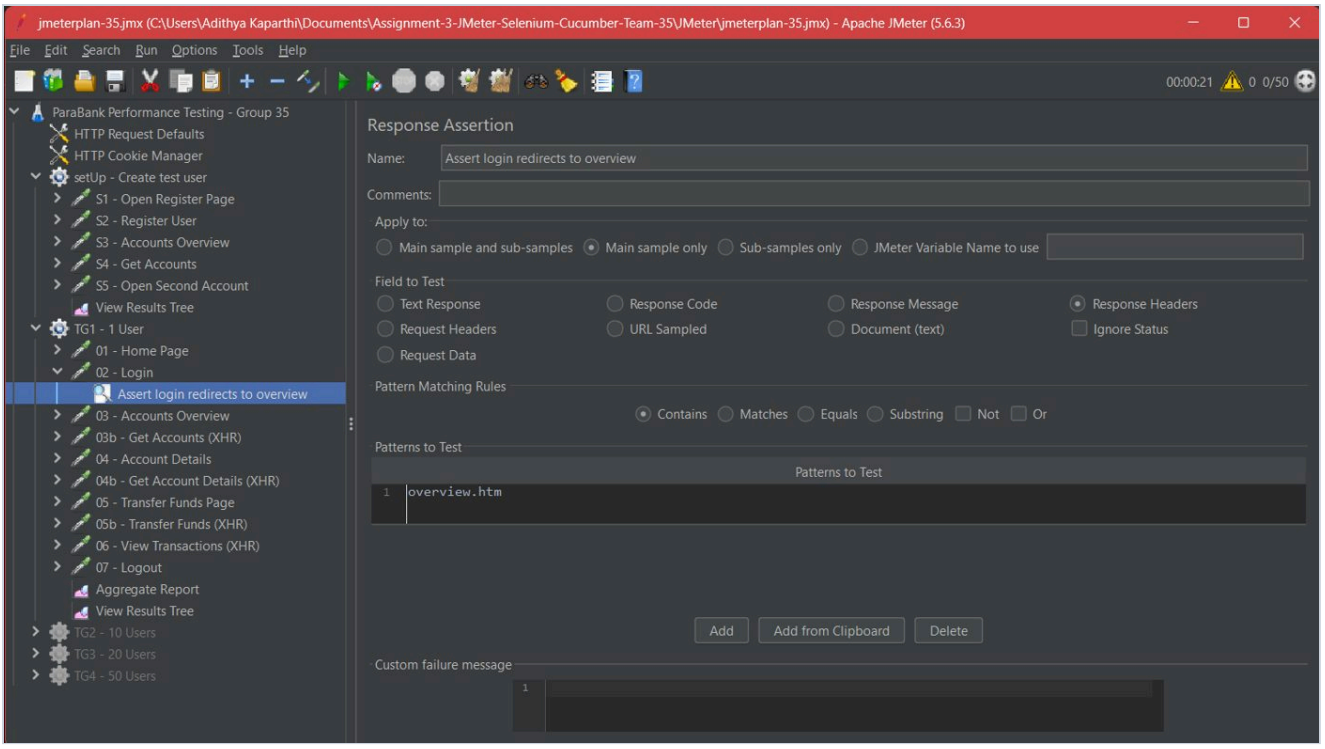


Figure A5. Original Login assertion: Response Headers contain `overview.htm`.

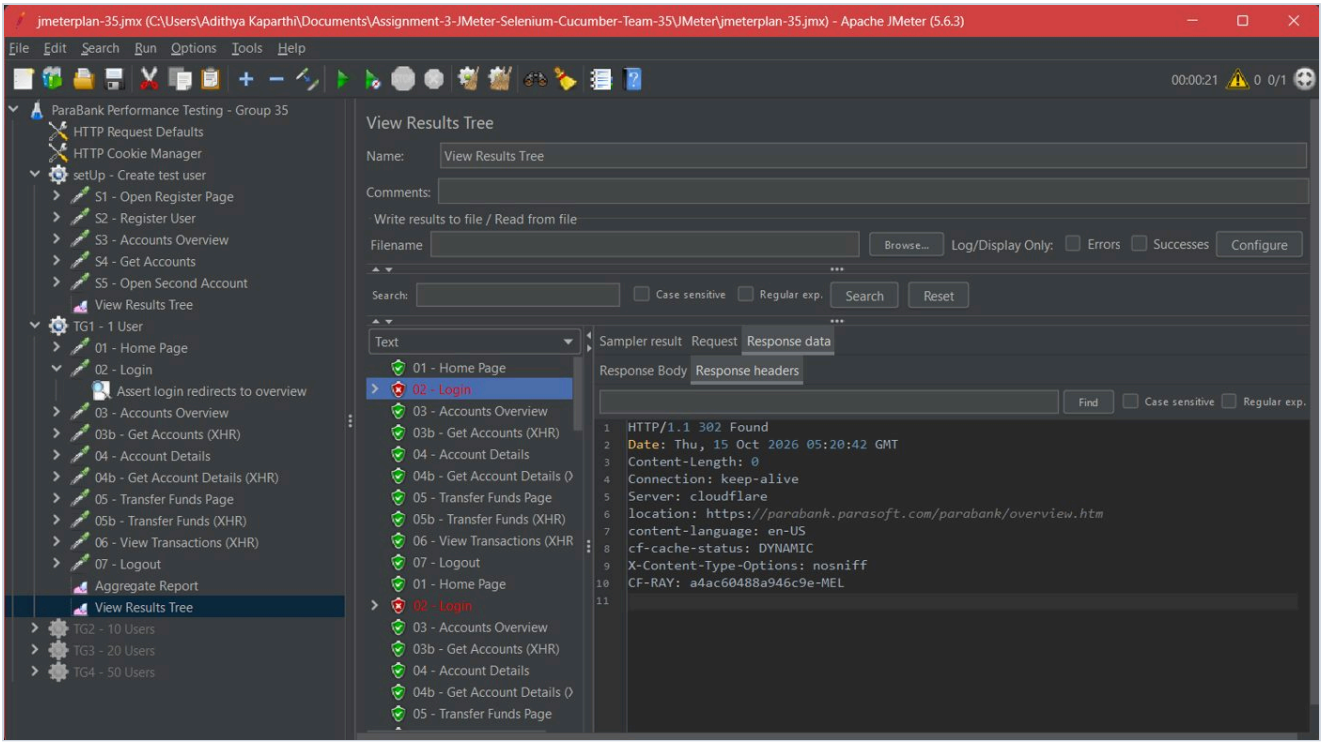


Figure A6. During the assertion test the server still returned `302 → overview.htm`.